

GRAPH COLORING USING BACKTRACKING

Mini Project Technical Report

Name: Santhosh Kumar G

Subject: Design and Analysis of Algorithms

Reg no: 711524BCS155

1. Introduction

Graph coloring is a classical problem in computer science and discrete mathematics. It involves assigning colors to vertices of a graph such that no two adjacent vertices share the same color. This problem has real-world applications in scheduling, register allocation in compilers, map coloring, and frequency assignment in mobile networks. The complexity of this problem increases with the number of vertices and edges. In this project, we use the backtracking technique to systematically explore all possible color assignments and ensure that constraints are satisfied.

2. Objectives

The main objective of this project is to understand and implement the backtracking technique using the graph coloring problem. The project also aims to apply constraints effectively while assigning colors, analyze the performance of recursive algorithms, and understand how exponential time complexity arises in such problems. Additionally, it helps in improving problem-solving and logical thinking skills.

3. Problem Statement

The problem is to color a graph consisting of 7 vertices using exactly 3 colors such that no two adjacent vertices have the

same color. The graph is represented using an adjacency matrix. The solution must ensure that all constraints are satisfied and should be implemented using a backtracking approach.

4. Algorithm

The algorithm begins by selecting the first vertex and attempting to assign one of the available colors. For each color assignment, a safety check is performed to ensure that no adjacent vertex has the same color. If the assignment is valid, the algorithm proceeds to the next vertex recursively. If no valid color can be assigned to a vertex, the algorithm backtracks to the previous vertex and tries a different color. This process continues until all vertices are successfully colored or all possibilities are exhausted.

5. Pseudocode

The pseudocode defines a recursive function that attempts to assign colors to each vertex. It checks whether a color is safe before assigning it. If a conflict occurs, the function removes the assigned color and tries the next one. This ensures that all possible combinations are explored systematically.

6. Implementation Details

The implementation is done using Java programming language. The graph is represented as a 2D adjacency matrix. An array is used to store the colors assigned to each vertex. The `isSafe` function checks whether assigning a particular color to a vertex violates any constraints. The `solve` function uses recursion and backtracking to explore all possible solutions. If a valid coloring is found, it is displayed as output.

7. Test Cases

Multiple test cases are used to validate the correctness of the algorithm. In the normal case, the graph is successfully colored using 3 colors. In the edge case, when only 2 colors are provided, the algorithm correctly identifies that no solution exists. In a larger case with more vertices, the algorithm still works but takes more time due to increased complexity.

8. Sample Input & Output

The input is provided in the form of an adjacency matrix representing the graph. The output displays each vertex along with its assigned color. The output ensures that no two adjacent vertices share the same color, satisfying the constraints of the problem.

9. Complexity Analysis

The time complexity of the graph coloring problem using backtracking is $O(M^N)$, where M is the number of colors and N is the number of vertices. This is because for each vertex, all possible color assignments are tried. The space complexity is $O(N)$ due to the recursion stack and the array used to store colors. In the best case, the algorithm may find a solution quickly, while in the worst case, it explores all possible combinations.

10. Advantages

The backtracking approach guarantees that a valid solution will be found if it exists. It is simple to implement and clearly demonstrates the concept of recursion and constraint satisfaction. It is also flexible and can be adapted to solve similar problems such as Sudoku and N-Queens.

11. Limitations

The major limitation of this approach is its high time complexity. As the number of vertices increases, the number of possible combinations grows exponentially, making the algorithm inefficient for very large graphs. It is not suitable for real-time applications where quick solutions are required.

12. Conclusion

This project successfully demonstrates the application of the backtracking technique to solve the graph coloring problem. It highlights how recursive algorithms can be used to explore solution spaces and enforce constraints. Despite its limitations, backtracking remains an important technique for solving combinatorial problems.

13. References

Standard textbooks on Design and Analysis of Algorithms, lecture notes, and various online educational resources were referred to for understanding the concept and implementation of the graph coloring problem.